修整

给定一棵以 1 为根、含 n 个节点的树，所有节点的状态值初始为 0。一条路径的闭邻域，指路径上的点，以及与路径上至少一个点相邻的点；同一个点在一次操作中只处理一次。

共 q 次操作：$1\ u\ v\ w$ 将路径 u 到 v 的闭邻域中所有值加上 w；$2\ u\ v\ z$ 将同一范围内的值改为原值与 z 的较大值；$3\ x$ 查询以 x 为根的子树中所有值之和。

从 trim.in 读入：第一行 n, q，接着 $n-1$ 行树边，再接着 q 行操作。向 trim.out 输出每次查询的答案，每行一个整数。

$1 \le n, q \le 10^5$，点的编号在 $[1, n]$ 内，$1 \le w \le 10^5$，$1 \le z \le 10^{10}$。所有状态值和答案均在 64 位有符号整数范围内。时间限制 2 秒，空间限制 512 MB。

样例输入、输出分别如下（左为输入，右为输出）。

```
7 7
1 2
2 3
3 4
3 5
2 6
2 7
1 3 4 6
3 5
3 3
3 2
2 1 7 11
3 1
3 3
```

```
6
18
24
67
23
```

把路径上每个点的儿子都修改一次，再补上两端点的 LCA 和它的父亲，就能完成题目要求的修改。因为路径上除 LCA 以外的点，父亲都在路径上，它们会作为儿子被修改；路径外与路径相邻的点，也只会是这些儿子，或者 LCA 的父亲。每个点只有一个父亲，所以这样不会重复修改，LCA 是根时不处理它的父亲。

比如树上的边为 $1-2, 2-3, 2-4, 3-5$，要修改路径 $3-2-4$。节点 2 的儿子有 $3, 4$，节点 3 的儿子有 5，把它们改完以后，再补上 LCA 2 和它的父亲 1，就把需要修改的五个点都处理到了。

接下来考虑怎么快速修改路径上所有点的儿子。先做树链剖分，把所有重儿子放进一个数组，所有轻儿子放进另一个数组，两个数组都按父亲的 dfn 排序。同一个父亲的儿子放在一起，这样父亲的 dfn 是一段区间时，对应的儿子在两个数组里也分别是一段区间。

用前缀和记录每个父亲往数组里放了多少个点，就能求出这两段区间的端点。树链剖分把路径拆成 $O(\log n)$ 段，每一段都按这个办法修改儿子，再单独修改 LCA 和它的父亲即可。两个数组中都没有根，把根另外存一个位置就行。

查询子树也可以用这两个数组。注意到 x 子树中所有点的儿子，恰好就是这棵子树去掉 x；而子树在 dfn 上是连续的，所以同样能找到两个儿子区间，查出它们的和，再加上 x 的值，就是所求答案。

这样就剩下数组上的区间加、区间取最大值和区间求和，考虑 Segment Tree Beats。每个节点记录区间和、最小值、严格次小值以及最小值的个数，加法直接打标记即可。

这里的严格次小值，指第二小的不同数值。比如一段数是 $2, 2, 7$，最小值为 2、个数为 2，次小值为 7。如果把它们和 5 取最大，只有两个 2 需要改成 5，区间和直接增加 $(5-2)\times 2=6$；如果和 8 取最大，连 7 也要改，就继续往下处理。一般也是这样：z 不超过最小值时跳过，严格小于次小值时只改最小值，其余情况向下递归。

花园

有 n 个区域，第 i 个区域有一株颜色为 c_i 的植物，颜色编号在 $[1,k]$ 内。区域之间有 m 条带权无向道路，没有自环和重边，图不一定连通。

一次考察从 s 出发，只允许走代价不超过 w 的道路，考察全部可达区域。每种颜色单独计分：出现 cnt 株就贡献 2^{cnt}，把所有颜色的贡献相加，再对 998244353 取模。没有出现的颜色也贡献 1。

有 q 次考察，输入给出加密后的 u_i, v_i。令上一次答案为 $last$，第一次前为 0，真实参数为 $s_i = u_i \operatorname{xor} last$、$w_i = v_i \operatorname{xor} last$。

从 garden.in 读入：第一行 n, k, m, q，第二行 n 个颜色，接着 m 行每行给出道路两端点与代价，最后 q 行每行给出 u_i, v_i。向 garden.out 依次输出各次考察的得分，每行一个整数。

$2 \le n \le 10^5$，$1 \le k, q \le 10^5$，$1 \le m \le 2\times10^5$，边权和真实阈值在 $[1, 998244353)$ 内，$0 \le u_i, v_i < 2^{30}$。解密后的参数合法。时间限制 2 秒，空间限制 512 MB。

样例输入、输出分别如下（左为输入，右为输出）。

```
6 3 6 5
1 2 1 3 2 3
1 2 2
2 3 5
3 4 3
4 5 7
5 6 1
2 5 4
1 1
2 5
6 6
4 1
12 15
```

```
4
5
5
8
12
```

只走边权不超过 w 的道路时，能从 s 到达的点，就是 s 所在的连通块。注意到随着 w 增大，连通块只会合并，不会拆开，考虑用 Kruskal 重构树把合并过程记录下来。

把边按权值从小到大排序，原图的每个点作为一个叶子。每次遇到一条连接两个不同连通块的边，就新建一个点，把两个连通块的根接到它下面，点权记为这条边的权值；如果两端已经连通，就直接跳过。

这样每个新点都代表一次合并后的连通块，它下面的叶子就是这个块里的原图节点。沿着父亲往上走，点权不会变小，所以询问 (s, w) 时，从 s 开始倍增，找到点权不超过 w 的最高祖先即可。原图不连通就保留多棵树，同权边形成的连续合并也能一起跳过去。

接下来给每个点预处理答案。两个块合并以后，同一种颜色的数量要相加，不能把原来的两个得分直接相加，因此还需要记录每种颜色出现了多少次。没有出现的颜色也贡献 1，可以先把所有颜色的这 1 分合成 k，线段树只维护每种颜色比 1 多出来的分数。比如一种颜色出现两次，原本贡献 4，叶子只记 $4-1=3$；最后把整棵线段树的和加上 k，就得到了原来的得分。

这样没出现的颜色就记为 0，用动态开点线段树维护颜色即可。叶子同时记录这一种颜色的数量和额外分数，内部节点维护额外分数之和。单个点只有一种颜色，额外分数是 $2-1=1$，所以整个块的得分就是 $k+1$。

合并两个连通块时，同时合并它们的线段树。一边为空就返回另一边，到了同一种颜色的叶子，就把数量相加，重新计算贡献；其余位置递归合并左右儿子即可。比如两块的颜色数量是 $(1,0,1)$ 和 $(0,2,0)$，合并以后变成 $(1,2,1)$，得分为 $2+4+2=8$。

每次合并后把答案存下来，后面的询问只读这个数，不会再用旧线段树，所以直接在线段树上合并就行，不需要保留历史版本。最初一共建立 $O(n\log(k+1))$ 个节点，后续合并不再开点，全部线段树合并也是这个时间。

最后按输入顺序解密询问，倍增找到对应的点，输出它的答案并更新 $last$。强制在线只要求询问按顺序处理，重构树和答案仍然可以提前算好。

建树以后，每次询问只需要倍增往上跳，时间为 $O(\log n)$。

印谱

有 m 种符号，编号为 $0, 1, \ldots, m-1$。第 i 天允许使用的符号集合为 A_i。给定 q 个互不相同的非空印记 S_j，第 j 个印记从刻成日 d_j 起才能选择。空印记从第 1 天起一直可以选择。

要选择连续 n 天的印记 $B_1, \ldots, B_n$。每天的印记必须为空，或是给出的非空印记之一；必须已经刻成，且满足 $B_i \subseteq A_i$；相邻两天还要满足 $B_i \subseteq B_{i+1}$，已经出现的符号不能去掉。

求方案数对 998244353 取模的结果。两个方案不同，当且仅当至少有一天选择的印记不同。集合用二进制整数表示，第 b 位为 1 表示包含符号 b，空集合为 0。

从 epigraphy.in 读入：第一行 n, m, q，第二行 n 个整数表示 A_i，接着 q 行每行给出印记 s_j 和刻成日 d_j。若两个给定印记满足 $S_x \subseteq S_y$，保证 $d_x \le d_y$。向 epigraphy.out 输出一个整数。

$1 \le n \le 2 \times 10^5$，$1 \le m \le 16$，$1 \le q \le 2^m - 1$，$0 \le A_i < 2^m$，$1 \le s_j < 2^m$，$1 \le d_j \le n$。时间限制 1 秒，空间限制 512 MB。

样例输入、输出分别如下（左为输入，右为输出）。

```
3 2 3
3 1 3
1 1
2 1
3 1
```

```
8
```

先考虑每天做一次 DP。今天如果选印记 s，昨天就可以选 s 本身，或者它的任意子印记，把这些方案数相加就是今天的结果。用 SOS DP 可以同时求出所有印记的转移，但天数太多，这样只能拿部分分。

注意已经出现的符号不能去掉。比如某个符号第 5 天不能用，那它在前 5 天都不能出现，否则到了第 5 天也没办法删掉。因此，对每个符号找到最后一个不能用的日期，在那一天之后再加入它，就不会与后面的限制冲突。

一个印记包含的符号都能一直保留，而且它也已经刻成时，才可以第一次出现。把这一天记为 r_s，也就是 ``刻成日" 和 ``各个符号最后缺失日加一" 中的最大值；超过第 n 天的印记不用考虑。题目保证子印记更早刻成，所以子印记也不会比它更晚开始能用。

现在看怎么跳过逐日计算。记 $f_s(i)$ 为第 i 天选 s 的方案数，昨天已经选 s 的方案今天可以直接延续，因此每天新增加的，只有从真子印记变成 s 的那些方案。也就是说，只要把真子印记昨天的方案数相加，再按天累加，就得到了 f_s。

先看只有两个符号、所有印记从第一天就能用的情况。只含符号 0 的印记，到了第 i 天有 i 种方案，因为可以任选前 i 天中的一天第一次加入它；只含符号 1 时也一样。两个符号都含有的印记，可以从空印记或这两个单符号印记转过来，所以第 i 天新增 $1 + 2(i-1)$ 种方案。

把每天新增的数加起来，就是 $1 + 3 + 5 + \cdots + (2i-1) = i^2$。因此不用真的算 i 天，只要把式子 i^2 存下来，最后代入 n 就行。一般情况也是这样：空印记的方案数是常数，每增加一层符号就多做一次累加，所以每个印记都能存成一个次数不超过 m 的多项式。

接下来把每个印记的式子存成一组系数，方便计算累加。下面用 $C(i,k)$ 表示从 i 个位置中选 k 个的组合数，例如 $C(i,2)=i(i-1)/2$，于是刚才的式子可以写成

$$i^2 = i + 2C(i,2).$$

我们依次存 ``常数、i、$C(i,2)$、$C(i,3)$……." 前面的系数。比如 i^2 就存成 $(0,1,2)$，因为它没有常数项，i 的系数是 1，$C(i,2)$ 的系数是 2。

这种存法方便在于：把每天的 1 累加起来得到 i，把每天的 $i-1$ 累加起来得到 $C(i,2)$，把每天的 $C(i-1,2)$ 累加起来得到 $C(i,3)$。在刚才的例子里，三个真子印记的式子相加是 $1+2i$，存成 $(1,2)$；转移时先取昨天的值 $1+2(i-1)$，再累计到今天，得到 $i+2C(i,2)$，存成 $(0,1,2)$。所以累计这一步就是把系数整体向右移一格，更多项也按这个规则处理。

如果某个印记要晚几天才能用，还需要减去开始前已经累计的部分。比如双符号印记第 3 天才能用，而两个单符号印记从第一天就能用，刚算出的式子仍是 i^2，但它在第 2 天应该为 0，所以减去 2^2，得到 i^2-4。第 3 天代入后是 5，正好对应昨天的空印记 1 种、两个单符号印记各 2 种。一般就减去式子在第 r_s-1 天的值，这只会改变常数项；更早日期算出的值不用当作方案数。

最后按印记所含符号的个数从小到大计算。先用 SOS 把所有真子印记的系数逐项相加，再右移一格，最后调整常数项，就求出了当前印记的式子。这里用 SOS 做的是对子集求和：固定一个系数位置，把各个子集在这个位置上的数相加。没有给出的印记不参与转移，最后把所有能用的印记在第 n 天的结果相加即可。

同铸

有 n 块材料，第 i 块材料的纯度为正整数 a_i。每次试验给出 m，选择三个两两不同的有序下标 (u, v, w)，以及三个正整数 (x, y, z)，满足 $xyz = m$，把 a_u, a_v, a_w 分别乘以 x, y, z，其他位置不变。

试验后的稳定值为全部 n 个数的最大公约数。每次试验相互独立，都从原数组开始。对每个 m，求能达到的最大稳定值，以及达到最大值的方案数。

两个方案不同，当且仅当有序下标三元组不同，或有序乘数三元组不同。最大稳定值不取模，只有方案数对 998244353 取模。

从 isogeny.in 读入：第一行 n, q，第二行 n 个整数 a_i，接着 q 行每行一个 m。向 isogeny.out 输出 q 行，每行分别给出最大稳定值和方案数。

$3 \le n \le 2 \times 10^5$，$1 \le q \le 2 \times 10^5$，$1 \le a_i, m \le 10^6$。最大稳定值不超过 10^{12}。时间限制 1 秒，空间限制 512 MB。

样例输入、输出分别如下（左为输入，右为输出）。

```
4 4
2 4 6 18
3
9
18
36
```

```
2 72
6 12
6 36
6 72
```

先把每个数都除以原数组的 gcd，得到 b_i。这样所有 b_i 的 gcd 为 1，最后再把求出的最大值乘回原来的 gcd，方案数不用变。

考虑枚举一个 d，看能不能让所有数都被它整除。某个位置原来是 b_i，它还需要补的因子是 $d / \gcd(d, b_i)$，记为 h_i。比如 $d = 6$，原数为 2 时，至少还要乘一个 3；原数已经是 6 的倍数时，就不需要补。

所有 $h_i > 1$ 的位置都必须选中，所以这样的下标不能超过三个。把它们需要补的因子乘起来，记成 H，则 m 还必须是 H 的倍数。注意不同位置的因子要分别补，比如两个位置都缺一个 3，总共就需要两个 3，不能只算一次。

把判定和计数放到一个例子里看。设数组为 $(1, 2, 6, 6, 6)$，要求全部被 6 整除，前两个位置就分别需要乘 6 和 3，所以位置 $1, 2$ 必须选，已经要用掉的乘积是 18。如果这次给出的 $m = 72$，剩下的乘积就是 $72/18 = 4$。

先数位置。前两个位置已经确定，第三个可以选位置 $3, 4, 5$，一共有 3 种选择。选齐三个下标后，还要区分顺序，例如 $(1, 2, 3)$ 和 $(2, 1, 3)$ 是不同方案，三个下标共有 6 种排列，因此位置这一部分有 $3 \times 6 = 18$ 种。一般就是先选上那 t 个必须选的位置，再从剩下的 $n - t$ 个里补选 $3 - t$ 个，最后乘 6。

再数剩下的乘积怎么分。对已经排好顺序的三个位置，先分别放入它们必须补的因子，再把剩余的 4 分给三个乘数。所有分法为

$$(4, 1, 1),\ (1, 4, 1),\ (1, 1, 4),\ (2, 2, 1),\ (2, 1, 2),\ (1, 2, 2).$$

所以剩余乘数有 6 种分法，位置选择和因子分配可以任意搭配，这个例子一共有 $18 \times 6 = 108$ 种方案。这里有序下标和有序乘数都要区分，不能因为材料的数值相同就合并方案。

一般情况下，把剩余乘积记为 v，要数的就是三个正整数相乘等于 v 的方法数。分解质因数后，分别考虑每一种质数即可：如果有 e 个这样的因子，第一个乘数拿 0 个时，后两个有 $e+1$ 种分法；拿 1 个时有 e 种，依次减少，总共是 $(e+1)(e+2)/2$ 种。各个质数分别算完再相乘，就是 v 的分法数，可以先预处理出来。

接下来优化枚举 d 时的计算。用倍数筛求出 $s[x]$，表示有多少个 b_i 能被 x 整除，那么必须修改的位置数就是 $n-s[d]$，超过三个就跳过。还需要快速算出 H 中每个质因子一共缺了多少个，这也能用 s 来算。

比如要求被 4 整除，原数是 $1,2,4$，三个位置分别缺两个、一个、零个因子 2，总共缺三个。也可以分两次统计：不被 2 整除的有一个，不被 4 整除的有两个，加起来仍是三个。一般对 d 的质因子 p，依次加上 $n-s[p]$、$n-s[p^2]$，直到 d 所含的最高次幂，就得到了 H 中这个质因子的个数。

由于所有 b_i 的 gcd 为 1，每种质因子都至少有一个位置完全没有，需要把 d 中这一部分全补上，因此 d 一定整除 H，而 H 又要整除 m。所以 d 只需枚举到最大查询值，算出 H 后，对它的所有倍数更新最大答案和方案数即可。

按 d 从小到大更新，后来的结果直接覆盖之前的结果。计数时只要求 gcd 是 d 的倍数，但最终留下的 d 已经最大，如果某个方案的实际 gcd 更大，那个更大的值也会被枚举到，所以最后保存的计数就是最优方案数。

首尾

有 n 项任务，原顺序为 $1, 2, \ldots, n$。第 i 项任务需要 t_i 单位时间，截止时间为 d_i。所有任务从时刻 0 开始依次执行，中途不休息，任务 i 在时刻 C_i 完成时，逾期时间为 $\max(0, C_i - d_i)$。

执行前可以选择一项任务，把它从原顺序中取出，放到所有任务的最前面或最后面，也可以不调整。求所有任务逾期时间的最大值最少是多少。

从 ends.in 读入：第一行 n，接下来 n 行每行给出 t_i, d_i。向 ends.out 输出一个整数，表示最小的最大逾期时间。

$1 \le n \le 2 \times 10^5$，$1 \le t_i \le 10^9$，$0 \le d_i \le 10^{14}$。时间限制 1 秒，空间限制 512 MB。

样例输入、输出分别如下（左为输入，右为输出）。

```
6
5 30
4 24
7 33
6 4
1 12
8 3
```

```
21
```

考虑枚举被移动的任务，其他任务的相对顺序不变，所以只要算出各段的完成时间增加或减少了多少，就能求出这次移动的结果。记 s_i 为前 i 项任务的耗时之和，原来任务 i 的完成时间就是 s_i，再用 $a_i = s_i - d_i$ 表示它与截止时间的差，先保留负数。

先预处理 a 的前缀最大值和后缀最大值，分别记为 pre_i 和 suf_i。这样想知道某一段里最晚超时的任务，只要取对应的前缀或后缀最大值，不用再逐个枚举。

把任务 i 放到最前面时，它自己在 t_i 时完成，是否逾期看 $t_i - d_i$；原来排在它前面的任务都多等了 t_i，这一段的最大差值就是 $pre_{i-1} + t_i$；原来排在它后面的任务，完成前还是做了同样的那些任务，所以完成时间不变，直接用 suf_{i+1}。把这三个值和 0 取最大，就是这次移动的最大逾期时间。

比如任务耗时依次为 $3, 5, 2$，把第二项移到最前面后，原来的第一项要多等 5，第二项自己在时刻 5 完成，第三项仍在总时间 10 完成。后面的任务没有变快或变慢，因为被移走的那项虽然换了位置，仍然排在它前面。

把任务 i 放到最后面时，原来在它前面的任务不受影响，这一段用 pre_{i-1}；原来在它后面的任务都提前了 t_i，这一段用 $suf_{i+1} - t_i$；它自己在全部任务做完的时刻 s_n 完成，用 $s_n - d_i$。仍然把三个值和 0 取最大，就得到了放到最后的结果。

这样每个 i 都只需要算几个数，枚举所有任务后，再与原顺序的答案取最小即可。注意前后缀最大值里要保留负数，例如某项任务原来能提前 5 个单位时间完成，多等 3 以后仍不逾期；如果先把负数改成 0，就会把这 3 个单位时间误算成逾期。预处理和枚举各扫一遍，时间复杂度为 $O(n)$。